

En la sesión anterior representamos solicitudes mediante diccionarios. Cuando el sistema crece, surgen problemas estructurales:

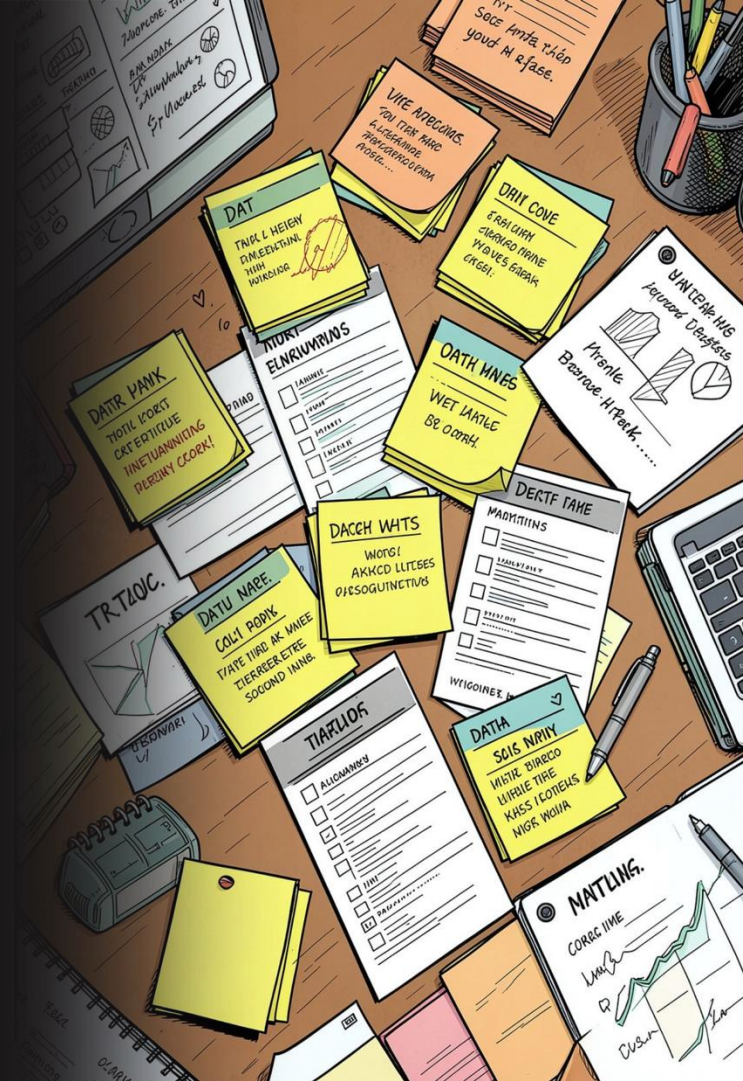
La lógica está dispersa por todo el programa.

Cualquier parte del código puede alterar los datos.

Se duplica comportamiento en distintos puntos.

No está claro qué componente hace qué.

⑦ ¿Dónde debería estar la lógica que permite cerrar una solicitud?



La clase define; el objeto representa

La clase como plantilla

Una **clase** establece qué datos tendrá un elemento, qué operaciones podrá realizar y qué reglas deberá respetar. Un **objeto** es una instancia concreta con su propio estado.

```
class Ticket:
    def __init__(
        self, ticket_id: int,
        title: str, priority: str,
    ):
        self.ticket_id = ticket_id
        self.title = title
        self.priority = priority
        self.status = "Pendiente"

ticket_1 = Ticket(
    1001, "Error al iniciar sesión", "Alta"
)
```

Mensaje clave

La clase describe una **estructura**; cada objeto contiene su propio **estado**.

Un objeto combina estado y comportamiento

Atributos: el estado

- Identificador, título y prioridad
- Estado y técnico asignado

Métodos: el comportamiento

- `assign()` — Asignar técnico
- `close()` — Cerrar solicitud
- `get_summary()` — Resumen del ticket

```
class Ticket:
    def assign(self, technician: str) -> None:
        self.technician = technician
        self.status = "Asignado"

    def close(self) -> None:
        self.status = "Cerrado"

    def get_summary(self) -> str:
        return (
            f"{self.ticket_id} - "
            f"{self.title} - "
            f"{self.status}"
        )
```

❏ Un objeto no debería ser un contenedor pasivo de información.

Proteger el estado interno del objeto

El **encapsulamiento** centraliza las reglas de modificación y evita estados inválidos.

```
class Ticket:
    def __init__(self, ticket_id: int, title: str):
        self.ticket_id = ticket_id
        self.title = title
        self._status = "Pendiente"

    @property
    def status(self) -> str:
        return self._status

    def close(self) -> None:
        if self._status == "Cerrado":
            raise ValueError("Ya está cerrada")
        self._status = "Cerrado"
```

Beneficios del encapsulamiento

Reglas centralizadas

Toda la lógica de validación vive en un único lugar.

Estados inválidos imposibles

Las transiciones incorrectas se previenen con excepciones.

Mantenimiento sencillo

Los cambios afectan solo a la clase, no a sus consumidores.

Mostrar lo necesario y ocultar la implementación

Una **abstracción** define qué operación debe existir sin obligar al consumidor a conocer sus detalles internos.

Contrato abstracto

```
from abc import ABC, abstractmethod

class NotificationChannel(ABC):
    @abstractmethod
    def send(
        self, recipient: str, message: str
    ) -> None:
        pass
```

Implementación concreta

```
class EmailNotification(NotificationChannel):
    def send(
        self, recipient: str, message: str
    ) -> None:
        print(
            f"Correo enviado a {recipient}: {message}"
        )
```

¿Por qué importa?

El sistema conoce la operación `send()`, pero ignora cómo se realiza el envío. Esto permite añadir nuevos canales (SMS, consola, webhook) sin modificar el código que los usa.



El sistema conoce la operación, no los detalles de su implementación.

Especializar comportamientos existentes

Relación "es un"

```
class User:
    def __init__(self, name: str, email: str):
        self.name = name
        self.email = email

class Technician(User):
    def __init__(
        self, name: str, email: str, specialty: str
    ):
        super().__init__(name, email)
        self.specialty = specialty

    def attend_ticket(self, ticket: Ticket) -> None:
        ticket.assign(self.name)
```

Technician es un **User** con comportamiento adicional.

Ventajas y riesgos

✓ Reutilización

Las características comunes se heredan automáticamente.

✓ Polimorfismo

Las subclases pueden usarse donde se espera la clase base.

⚠ Jerarquías profundas

Demasiada herencia genera acoplamiento rígido y difícil de mantener.

Construir objetos a partir de otros objetos

La composición representa una relación **"tiene un"** y ofrece mayor flexibilidad que la herencia.

Ejemplo: TicketService

```
class TicketService:
    def __init__(
        self, notification: NotificationChannel
    ):
        self.notification = notification

    def register(
        self, ticket: Ticket, requester_email: str
    ) -> None:
        message = f"Solicitud {ticket.ticket_id} registrada"
        self.notification.send(
            requester_email, message
        )

service = TicketService(EmailNotification())
```

Herencia vs. Composición

Herencia	Composición
"Es un"	"Tiene un"
Relación rígida	Relación flexible
Especialización	Colaboración
Clases relacionadas	Objetos intercambiables



Preferir composición cuando el comportamiento deba poder cambiarse.

Cinco principios para reducir el costo del cambio

Principio	Idea principal	Ejemplo
S — SRP	Una clase, un motivo para cambiar	Ticket no envía correos
O — Abierto/cerrado	Extender sin modificar	Agregar SMSNotification
L — Liskov	Respetar el contrato base	Todo canal ejecuta send()
I — Segregación	Contratos pequeños y específicos	Sin métodos innecesarios
D — Inversión	Depender de abstracciones	TicketService usa NotificationChannel

Ejemplo incorrecto (viola SRP)

```
class Ticket:
    def save_database(self): pass
    def send_email(self): pass
    def generate_pdf(self): pass
```

Estructura del proyecto

```
helpdesk/
├─ main.py
├─ domain/
│   ├─ ticket.py
│   └─ user.py
├─ services/
│   └─ ticket_service.py
├─ notifications/
│   ├─ base.py
│   ├─ email.py
│   └─ console.py
└─ policies/
    └─ response_time.py
```

- ❑ **Strategy:** cambia reglas sin modificar el consumidor. **Factory:** centraliza la creación de objetos. Un patrón es una solución reutilizable, no código para copiar.



Construye una solución orientada a objetos

La empresa requiere registrar solicitudes y notificar al usuario. El canal de notificación puede cambiar sin modificar el servicio principal.

01

Clase Ticket

Atributo encapsulado para el estado, métodos `assign()` y `close()`.

02

Abstracción NotificationChannel

Implementación `EmailNotification` y composición en `TicketService`.

03

Separación en módulos

Responsabilidades claras, nombres descriptivos, tipos correctos y excepciones específicas.



¿Qué deberíamos modificar para agregar notificaciones por SMS?